Laporan Ujian Tengah Semester Graf Sains Data

Python Driver untuk Neo4j dan Sistem Rekomendasi (RetailRocket)

https://github.com/sisiliams77/UTS-GRAF-SAINS-DATA.git

Nama: Maria Sisilia S

NIM: 23031554168

1. Tugas Pendahuluan

- Persiapan dan Perkenalan Lingkungan
  Langkah yang dikerjakan: membuat environment baru, mengaktifkannya, dan memasang dependensi. Pada sel pertama notebook, keluaran `neo4j driver: 6.4.1` memastikan versi driver memenuhi syarat.
- Tutorial Driver Neo4j
  Tutorial membuat node `Person` dan relasi `KNOWS` dengan perintah `MERGE` melalui `driver.execute_query`, lalu membaca kembali dengan `RoutingControl.READ`. Hasil eksekusi:

```
Guinevere
Lancelot
Merlin
```

Setelah tutorial, isi database dikosongkan dengan `MATCH (n) DETACH DELETE n` (keluaran: "Database dikosongkan.").

```
# Reset isi database setelah tutorial
with GraphDatabase.driver(URI, auth=AUTH) as d:
    d.execute_query("MATCH (n) DETACH DELETE n", database_=DB)
print("Database dikosongkan.")
```

```
Database dikosongkan.
```

Jawaban Pertanyaan Pendahuluan (Checkpoint)

1) **Apa nama database yang digunakan pada tutorial ini?**
   Database `neo4j` (database default), yang diberikan melalui argumen `database_="neo4j"` pada `execute_query`.
2) **Akses lain untuk mengakses database lain dalam satu instance?**

- Python driver: driver.session(database="namadb") atau driver.execute_query(..., database_="namadb"). Operasi tingkat instance seperti CREATE DATABASE dan SHOW DATABASES dijalankan pada database system.
- Cypher: klausa USE namadb (di Neo4j Browser atau cypher-shell, :use namadb), misalnya USE otherdb MATCH (n) RETURN n.
- DDL: CREATE DATABASE namadb, SHOW DATABASES, dan DROP DATABASE namadb.
- Banyak database pengguna dalam satu instance memerlukan Enterprise Edition; Community Edition hanya memiliki neo4j dan system.

3) **Jenis graf pertemanan yang dibuat?**
Property graph berarah (directed), unipartite (satu label, yaitu Person), dan tidak berbobot, dengan satu jenis relasi KNOWS. Strukturnya berbentuk star atau ego-network: Arthur menjadi hub dengan out-degree 3, total 4 simpul dan 3 sisi, tanpa siklus (pohon berarah). Penggunaan MERGE menjaga agar simpul dan relasi tidak terduplikasi bila perintah dijalankan ulang.

4) **Persiapan Data: RetailRocket**

Dataset RetailRocket (sumber: Roman Zykov) diunduh dari Kaggle dan hanya berkas events.csv yang dipakai. Fitur yang ada:

Kolom	Keterangan
timestamp	Waktu interaksi dalam epoch milidetik
visitorid	ID unik pengunjung
event	Jenis interaksi: view, addtocart, transaction
itemid	ID unik produk
transactionid	ID transaksi (hanya terisi pada event transaction)

Soal UTS Wrangling dan EDA

1. Penentuan Dua Periode Waktu

Eksplorasi awal terhadap events.csv:

Ukuran	Nilai
Jumlah baris	2.756.101
Rentang waktu	3 Mei 2015 s.d. 18 September 2015 (UTC)
Visitor unik	1.407.580
Item unik	235.061

```
shape: (2756101, 6)
rentang: 2015-05-03 03:00:04.384000+00:00 -> 2015-09-18 02:59:47.788000+00:00
event
view           2664312
addtocart        69332
transaction      22457
Name: count, dtype: int64

visitor unik: 1407580 | item unik: 235061
datetime
2015-05    590652
2015-06    610393
2015-07    697984
2015-08    553362
2015-09    303710
Name: jumlah_event, dtype: int64
```

Distribusi event: `view` 2.664.312, `addtocart` 69.332, dan `transaction` 22.457. Jumlah event per bulan: Mei 590.652; Juni 610.393; Juli 697.984; Agustus 553.362; September 303.710.

Dua periode yang digunakan (kolom `timestamp` dikonversi dengan `pd.to_datetime(unit="ms", utc=True)`, batas akhir bersifat eksklusif):

Fold	Peran	Periode
P1	Training	15 Mei 2015 s.d. 8 Agustus 2015
P2	Testing	12 Agustus 2015 s.d. 29 Agustus 2015

2 : Filter Periode dan Penyimpanan CSV

Filter diterapkan pada kolom `timestamp`, kemudian hasilnya disimpan ke `data/events_p1.csv` dan `data/events_p2.csv`.

Fold	Baris	Visitor unik	Item unik
P1 (training)	1.798.360	921.303	195.541
P2 (testing)	307.624	175.333	82.603

Hanya P1 yang diimpor ke Neo4j sebagai subgraph training. P2 dipakai sebagai ground truth evaluasi

3: Impor Data P1 ke Neo4j

Impor dilakukan per batch (5.000 baris per batch, 360 batch) dengan `UNWIND` dan `MERGE`. Constraint keunikan dibuat terlebih dahulu agar `MERGE` efisien:

```python
def create_constraints(session):
    session.run("CREATE CONSTRAINT visitor_id IF NOT EXISTS FOR (v:Visitor) REQUIRE v.visitorId IS UNIQUE")
    session.run("CREATE CONSTRAINT item_id IF NOT EXISTS FOR (i:Item) REQUIRE i.itemId IS UNIQUE")
```

Query impor tiap batch:

```
UNWIND $batch AS row
MERGE (v:Visitor {visitorId: row.visitorid})
MERGE (i:Item {itemId: row.itemid})
FOREACH (_ IN CASE WHEN row.event = 'view' THEN [1] ELSE [] END |
    CREATE (v)-[:VIEWED {timestamp: row.timestamp}]->(i))
FOREACH (_ IN CASE WHEN row.event = 'addtocart' THEN [1] ELSE [] END |
    CREATE (v)-[:ADDED_TO_CART {timestamp: row.timestamp}]->(i))
FOREACH (_ IN CASE WHEN row.event = 'transaction' THEN [1] ELSE [] END |
    CREATE (v)-[:BOUGHT {timestamp: row.timestamp, transactionId: row.transactionid}]->(i))
"""
```

Versi FOREACH + CASE ini menggantikan CALL { ... UNION ALL ... } pada contoh soal dan memberikan hasil yang setara: satu relasi dibuat untuk setiap event sesuai jenisnya. Proses impor selesai dengan pesan "Proses impor berhasil."

4: Komposisi Subgraph

Komposisi dihitung dengan Python API (driver.execute_query) menggunakan dua query Cypher:

```
MATCH (n) RETURN labels(n)[0] AS jenis_simpul, count(n) AS jumlah ORDER BY jumlah DESC

MATCH ()-[r]->() RETURN type(r) AS jenis_relasi, count(r) AS jumlah ORDER BY jumlah DESC
```

Penjelasan query: MATCH (n) mencocokkan seluruh simpul; labels(n)[0] mengambil label simpul; count(n) menghitung simpul per label (pengelompokan implisit berdasarkan label); MATCH ()-[r]->() mencocokkan seluruh relasi berarah; type(r) memberi jenis relasi; dan ORDER BY jumlah DESC mengurutkan dari yang terbanyak.

Hasil:

Jenis simpul	Jumlah
Visitor	921.303
Item	195.541
Total	**1.116.844**

Jenis relasi	Jumlah
VIEWED	1.739.212
ADDED_TO_CART	44.482
BOUGHT	14.666
Total	**1.798.360**

Total relasi sama dengan jumlah baris P1 (1.798.360), yang menunjukkan setiap event terimpor tepat satu kali. Graf bersifat bipartit (Visitor ke Item) dengan dominasi relasi VIEWED (sekitar 96,7%).

5: VSM terhadap Simpul User

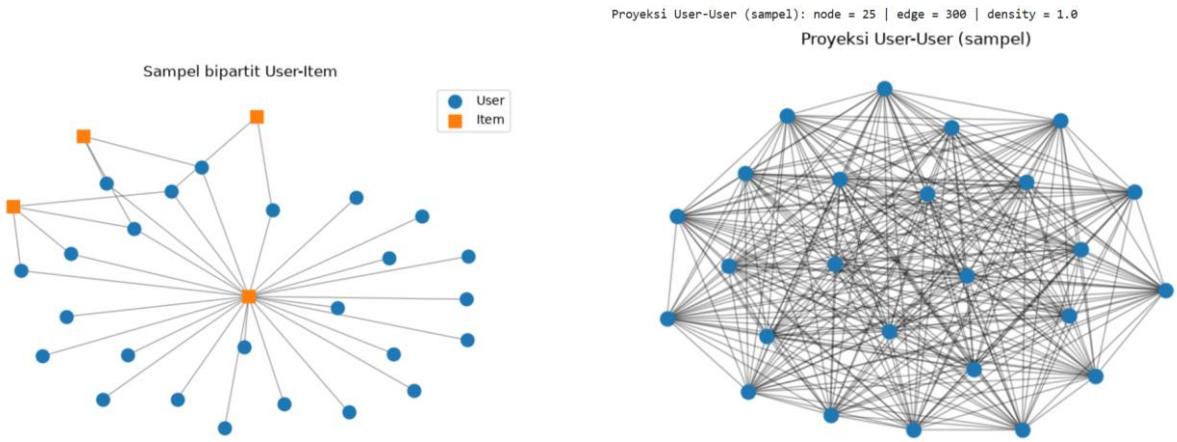
Graf bipartit diubah menjadi matriks interaksi **A** (user × item). Nilai sel adalah jumlah bobot interaksi (view = 1, addtocart = 3, transaction = 5) yang diambil dengan query agregasi:

```
Q_EDGES = """
MATCH (v:Visitor)-[r:VIEWED|ADDED_TO_CART|BOUGHT]->(i:Item)
RETURN v.visitorId AS visitorId, i.itemId AS itemId,
       sum(CASE type(r) WHEN 'VIEWED' THEN $wv WHEN 'ADDED_TO_CART' THEN $wa ELSE $wb END) AS w
"""
```

Setiap user direpresentasikan sebagai vektor pada ruang item (satu baris matriks A). Karakteristik VSM User:

Karakteristik	Nilai
Jumlah vektor (user)	921.303
Dimensi vektor (jumlah item)	195.541
Shape	(921.303, 195.541)
Elemen tidak nol (nnz)	1.398.277
Density	0,000008
Sparsity	99,9992%
Rata-rata nnz per user	1,52
Median nnz per user	1
Maksimum nnz per user	2.838
Persentase user dengan 1 nnz	79,94%

Plot networkx pada sampel (25 user dan 4 item dengan 34 sisi, dipilih secara reproducible dari pengunjung sebuah item populer) memperlihatkan struktur bipartit. Proyeksi User-User pada sampel memiliki 25 node, 300 sisi, dan density 1,0, yaitu graf lengkap.



gambar: plot bipartit sampel dan plot proyeksi User-Use

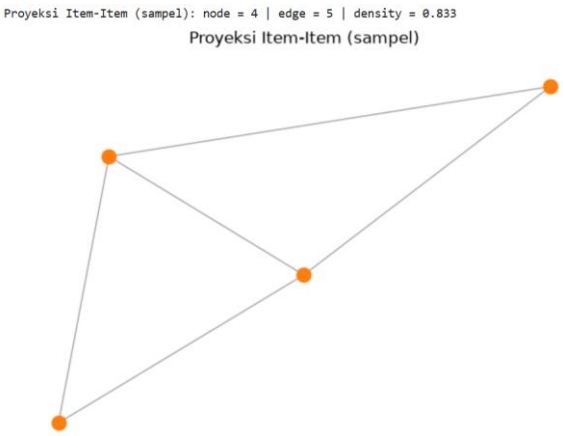
Interpretasi: vektor user berdimensi sangat tinggi (195.541) dan hampir seluruhnya bernilai nol. Hampir 80% user hanya berinteraksi dengan satu item (kasus cold-start), sehingga vektornya hampir tidak informatif untuk membandingkan kemiripan antar user. Pada sampel, proyeksi User-User menjadi graf lengkap karena seluruh user sampel berbagi satu item populer; hal ini menunjukkan item populer membuat proyeksi User-User meledak menjadi klik besar.

6: VSM terhadap Simpul Item

Setiap item direpresentasikan sebagai vektor pada ruang user (satu baris matriks transpose $\mathbf{A}^{\mathbf{T}}$).

Karakteristik	Nilai
Jumlah vektor (item)	195.541
Dimensi vektor (jumlah user)	921.303
Shape	(195.541, 921.303)
Elemen tidak nol (nnz)	1.398.277
Density	0,000008
Sparsity	99,9992%
Rata-rata nnz per item	7,15
Median nnz per item	2
Maksimum nnz per item	1.736
Persentase item dengan 1 nnz	37,94%

Proyeksi Item-Item pada sampel memiliki 4 node, 5 sisi, dan density 0,833.



Gambar: plot proyeksi Item-Item

Interpretasi: nnz total dan sparsity sama dengan VSM User karena keduanya berasal dari matriks yang sama. Namun distribusinya berbeda: vektor item lebih padat (rata-rata 7,15 interaksi per item, median 2) dan hanya 37,94% item yang memiliki satu interaksi, jauh lebih baik daripada VSM User (79,94%). Item juga memiliki ekor distribusi panjang, dengan item terpopuler memiliki 1.736 user.

Proyeksi yang Lebih Efektif dan Efisien

Biaya proyeksi diukur dengan jumlah node, jumlah wedge (pasangan simpul yang berbagi tetangga, batas atas jumlah sisi proyeksi), jumlah pasangan penuh, dan memori matriks similarity penuh (float32):

Proyeksi	Node	Wedge	Pasangan penuh n(n-1)/2	Memori matriks penuh
User-User	921.303	41.061.668	$4{,}24 \times 10^{11}$	sekitar 3.395 GB
Item-Item	195.541	19.787.023	$1{,}91 \times 10^{10}$	sekitar 153 GB

Kesimpulan: **proyeksi Item-Item lebih baik**.

- Efisien: jumlah node 4,7 kali lebih sedikit, wedge 2,1 kali lebih sedikit, dan memori matriks similarity penuh sekitar 22 kali lebih kecil. Karena matriks penuh Item-Item pun masih sekitar 153 GB, similarity tidak dihitung penuh melainkan secara top-K untuk entitas terpilih.
- Efektif: vektor item lebih informatif (rata-rata 7,15 interaksi, hanya 37,94% bernilai tunggal) dibanding vektor user (rata-rata 1,52, 79,94% bernilai tunggal), sehingga cosine similarity antar item lebih stabil.

7: Perhitungan Similarity pada Graf Terproyeksi

Similarity dihitung dengan cosine pada vektor item yang telah dinormalisasi L2. Karena matriks penuh tidak praktis, similarity dihitung untuk 500 item terpopuler (ditambah item acuan) dengan 10 tetangga terdekat masing-masing, sehingga terbentuk 5.000 pasangan. Item

acuan adalah item dengan derajat tertinggi, yaitu **Item 5411**. Skor ditulis kembali ke graf sebagai relasi :SIMILAR:

```
UNWIND $rows AS row
MATCH (a:{C['label']} {{{C['key']}: row.a}}), (b:{C['label']} {{{C['key']}: row.b}})
MERGE (a)-[s:SIMILAR]->(b)
SET s.score = row.score, s.method = 'cosine'
```

(a) Cypher untuk 5 record dengan similarity tertinggi (acuan: Item 5411):

```
MATCH (a:{C['label']} {{{C['key']}: $ref}})-[s:SIMILAR]->(b:{C['label']})
RETURN a.{C['key']} AS acuan, b.{C['key']} AS tetangga, s.score AS similarity
ORDER BY similarity DESC LIMIT 5
```

(b) Hasil query pada graf yang telah diperbarui:

Acuan	Tetangga	Similarity
5411	33030	0,148574
5411	37807	0,103283
5411	33956	0,102651
5411	256242	0,087303
5411	459936	0,083055

Skor similarity tertinggi hanya sekitar 0,15. Hal ini wajar karena vektor item berdimensi sangat tinggi dan sangat jarang: dua item jarang dikunjungi oleh sekumpulan user yang sama.

8: Kondisi Sebelum dan Sesudah Perhitungan Similarity

Query untuk Neo4j Browser (ganti <REF_ID> dengan 5411):

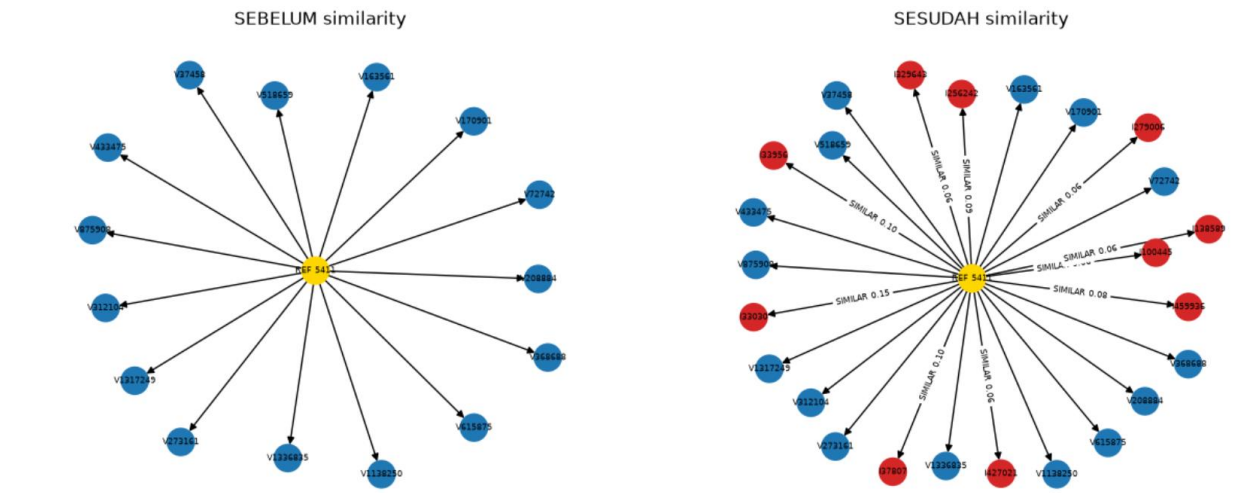
```
// SEBELUM: tetangga bipartit dari record acuan (tanpa SIMILAR)
MATCH p=(a {itemId: <REF_ID>})-[:VIEWED|ADDED_TO_CART|BOUGHT]-(o) RETURN p LIMIT 25

// SESUDAH: ditambah relasi SIMILAR
MATCH p=(a {itemId: <REF_ID>})-[:SIMILAR]->(b) RETURN p
```

Perbandingan jumlah relasi:

Relasi	Sebelum	Sesudah
VIEWED	1.739.212	1.739.212
ADDED_TO_CART	44.482	44.482
BOUGHT	14.666	14.666

Relasi	Sebelum	Sesudah
SIMILAR	0	5.000



Gambar: plot networkx SEBELUM similarity dan SESUDAH similarity

Interpretasi: relasi interaksi tidak berubah; satu-satunya perbedaan adalah munculnya 5.000 relasi `SIMILAR` berbobot yang menghubungkan item dengan tetangga terdekatnya. Pada gambar sesudah, item acuan terhubung ke tetangga dengan label skor similarity.

Soal 2.2 Analytics

1. Persiapan: Ground Truth dan Pemilihan User Acak

- Ground truth adalah item pada P2 yang belum pernah berinteraksi dengan user tersebut pada P1 dan yang ada di katalog item P1 (agar dapat direkomendasikan).
- Kandidat user adalah user pada matriks P1 yang memiliki minimal satu item ground truth: **11.022 user**.
- User dipilih dengan `np.random.default_rng(SEED).choice(...)`, `SEED = 42`, pada daftar kandidat yang telah diurutkan, sehingga hasilnya dapat direproduksi.

User terpilih: **123595**. User ini hanya memiliki 1 item pada P1 dan 17 item ground truth pada P2: 20623, 96680, 101196, 111831, 134436, 144513, 148730, 184217, 184620, 263258, 317978, 319192, 356048, 374698, 425186, 437929, 462070.

1a: Skrip Rekomendasi KNN (k = 5)

Mode rekomendasi mengikuti proyeksi terpilih (Item-Item), sehingga digunakan **item-based KNN**:

1. Ambil item yang pernah berinteraksi dengan user target pada P1 (maksimal 30 item teratas menurut bobot) sebagai item benih.

2. Untuk setiap item benih, cari k = 5 item tetangga terdekat berdasarkan cosine similarity pada vektor item.
3. Skor kandidat adalah akumulasi bobot interaksi item benih dikali similarity.
4. Item yang sudah pernah berinteraksi dengan user dikeluarkan, lalu kandidat diurutkan berdasarkan skor.

Karena user target hanya memiliki satu item pada P1, seluruh rekomendasi berasal dari lima tetangga terdekat satu item benih tersebut.

1b: Tiga Item yang Diiklankan (k = 5)

Daftar rekomendasi top-5:

Rank	Item ID	Skor
1	330894	0,862331
2	121304	0,545455
3	63338	0,545455
4	186780	0,545455
5	273223	0,545455

Tiga item yang diiklankan kepada user 123595 (tiga teratas): **330894, 121304, dan 63338**.

2: Evaluasi Hit@5 dan NDCG@5

Definisi: Hit@K bernilai 1 jika ada item ground truth di K teratas, selain itu 0. DCG@K = Σ rel_i / $\log_2(i+1)$ dengan rel_i = 1 bila item ke-i ada di ground truth, dan NDCG@K = DCG@K / IDCG@K. Evaluasi dihitung pada daftar top-5 yang sama dengan tiga item iklan (tiga teratas); nilai @3 ditampilkan sebagai pembanding.

Metrik	@5	@3
Hit	0	0
NDCG	0,0000	0,0000

Tidak ada satu pun item rekomendasi (330894, 121304, 63338, 186780, 273223) yang muncul pada ground truth user, sehingga Hit@5 = 0 dan NDCG@5 = 0.

Evaluasi tambahan pada 200 user acak (seed yang sama): rata-rata Hit@5 = 0,0050 dan rata-rata NDCG@5 = 0,0025, atau setara dengan satu user yang berhasil mendapat hit dari 200 user.

Pembahasan

Nilai metrik yang rendah dapat dijelaskan oleh karakteristik data:

- **Cold-start dan sparsity ekstrem.** 79,94% user hanya memiliki satu interaksi, dan user terpilih hanya memiliki satu item pada P1, sehingga sinyal preferensi sangat sedikit.

- **Ruang kandidat sangat besar.** Terdapat 195.541 item pada katalog P1, sedangkan hanya lima item yang direkomendasikan; peluang acak untuk mengenai 17 item ground truth sangat kecil.
- **Ground truth bersifat ketat.** Hanya item baru yang belum pernah berinteraksi pada P1 yang dihitung, dan perilaku pada P2 (periode berbeda) belum tentu mengikuti pola pada P1.
- **Similarity rendah.** Skor cosine item hanya sekitar 0,15 pada contoh item populer, sehingga sinyal tetangga lemah.

Perbaikan yang dapat dipertimbangkan: memakai fallback popularitas untuk user cold-start, menggabungkan jenis event dengan pembobotan berbeda, menambah jumlah item benih, atau memakai pendekatan berbasis faktorisasi matriks.

Kesimpulan

1. Neo4j dengan Python driver berhasil dipakai untuk mengimpor 1.798.360 event RetailRocket periode P1 menjadi graf bipartit dengan 1.116.844 simpul dan 1.798.360 relasi.
2. VSM menunjukkan data sangat jarang (sparsity 99,9992%) dengan mayoritas user hanya berinteraksi satu kali.
3. Proyeksi Item-Item dipilih karena lebih efisien (node, wedge, dan memori lebih kecil) dan lebih efektif (vektor lebih informatif).
4. Similarity cosine berhasil dihitung dan ditulis ke graf sebagai 5.000 relasi `SIMILAR`; item acuan 5411 memiliki tetangga terdekat 33030 dengan similarity 0,1486.
5. Rekomendasi KNN berbasis item untuk user 123595 menghasilkan Hit@5 = 0 dan NDCG@5 = 0; pada 200 user, Hit@5 = 0,0050 dan NDCG@5 = 0,0025. Hasil ini mencerminkan kesulitan rekomendasi pada data yang sangat jarang dan didominasi user cold-start.